

CKKS

Cheon–Kim–Kim–Song

8 avril 2026

Résumé

Le schéma CKKS (*Cheon–Kim–Kim–Song*) est un chiffrement homomorphe *approximatif* conçu pour effectuer des calculs sur des nombres réels ou complexes chiffrés. Contrairement aux schémas exacts, il accepte une petite erreur de représentation et d’arrondi, ce qui le rend plus adapté aux applications où une précision flottante suffit, par exemple en apprentissage automatique ou en calcul scientifique privé. Ce document présente les idées mathématiques de base ainsi que le fonctionnement du schéma.

Table des matières

1	Introduction	3
2	Fondements mathématiques	3
2.1	Anneaux de polynômes	3
2.2	Les idées de bruit, d’approximation et d’encodage	4
3	L’encodage CKKS	4
3.1	Des polynômes pour vectoriser	4
3.2	Une écriture simplifiée de l’encodage	4
4	Description détaillée de CKKS	5
4.1	Aperçu général	5
4.2	Génération des clés	5
4.3	Encodage et chiffrement	5
4.4	Déchiffrement	6
4.5	Addition homomorphe	6
4.6	Multiplication homomorphe	6
5	Bootstrapping	7
5.1	Le problème du bruit	7
5.2	Principe du bootstrapping	8
6	Limites	8
6.1	Précision approximative	8

6.2	Coût computationnel	8
6.3	Gestion du bruit	8
7	Conclusion	8

1 Introduction

Les premiers schémas homomorphes ne permettaient que des opérations très limitées. CKKS, introduit par Cheon, Kim, Kim et Song, est particulièrement intéressant car il travaille avec des nombres *approximatifs* au lieu d'exiger une exactitude parfaite [?].

L'intérêt principal de CKKS est qu'il permet de calculer sur des valeurs réelles ou complexes encodées sous forme de polynômes. En pratique, cela convient très bien à des domaines où l'on manipule déjà des données approchées : statistiques, filtrage, apprentissage automatique, calcul numérique, etc. Il y a cependant une petite erreur numérique toujours présente qui augmente au fil des opérations [?, ?].

2 Fondements mathématiques

2.1 Anneaux de polynômes

Un polynôme est une expression de la forme

$$p(X) = a_0 + a_1X + \cdots + a_dX^d,$$

où les coefficients a_i appartiennent à un ensemble donné, par exemple $\mathbb{Z}$ ou $\mathbb{Z}_q$.

Dans CKKS, on utilise un anneau du type

$$R_q = \mathbb{Z}_q[X]/(X^N + 1),$$

où :

- N est généralement une puissance de 2 ;
- $\mathbb{Z}_q$ désigne les entiers modulo q ;
- l'écriture modulo $(X^N + 1)$ signifie que deux polynômes sont considérés comme égaux dès que leur différence est multiple de $X^N + 1$.

Dans cet anneau, on fait les calculs comme d'habitude sur les polynômes, puis on réduit le résultat modulo $X^N + 1$ et modulo q .

Exemple 2.1 (Multiplication dans un anneau quotient). *Prenons*

$$R_{17} = \mathbb{Z}_{17}[X]/(X^2 + 1).$$

Calculons

$$(3 + 5X)(7 + 4X).$$

On développe :

$$(3 + 5X)(7 + 4X) = 21 + 12X + 35X + 20X^2 = 21 + 47X + 20X^2.$$

Dans l'anneau, on a $X^2 \equiv -1$, donc

$$20X^2 \equiv -20 \equiv 14 \pmod{17}.$$

Ainsi

$$21 + 47X + 20X^2 \equiv 21 + 47X + 14 \equiv 1 + 13X \pmod{17}.$$

On obtient donc

$$(3 + 5X)(7 + 4X) \equiv 1 + 13X \pmod{X^2 + 1, 17}.$$

Cet exemple montre l'idée essentielle : le calcul se fait d'abord comme sur les polynômes ordinaires, puis on ramène le résultat dans une forme standard grâce aux réductions modulo.

2.2 Les idées de bruit, d'approximation et d'encodage

Dans CKKS, un message chiffré n'est pas stocké de manière exacte. Il est représenté avec un petit *bruit* numérique ou algébrique, indispensable pour la sécurité.

Définition 2.1 (Encodage/Décodage). *On appelle encodage la transformation qui convertit un message numérique classique, généralement un vecteur de nombres réels ou complexes, en une représentation adaptée au schéma de chiffrement utilisé (CKKS).*

Inversement, le décodage est l'opération qui, à partir d'une donnée encodée, permet de retrouver une approximation du message original.

Définition 2.2 (Bruit). *On appelle bruit la petite erreur présente dans un chiffré et qui fait que le déchiffrement ne redonne pas directement la valeur encodée, mais une valeur très proche.*

Dans CKKS, on accepte ce bruit et on l'interprète comme une erreur d'approximation. L'idée est alors la suivante :

$$\text{valeur exacte} \longrightarrow \text{valeur encodée} \longrightarrow \text{chiffré}$$

puis, après calcul,

$$\text{chiffré calculé} \longrightarrow \text{valeur décodée approximative.}$$

Pour contrôler cette approximation, on introduit un *facteur d'échelle* Δ , souvent une grande puissance de 2. Les nombres réels sont multipliés par Δ avant d'être arrondis à des entiers. À la fin, on divise par Δ pour revenir à l'échelle initiale.

3 L'encodage CKKS

3.1 Des polynômes pour vectoriser

CKKS ne chiffre pas un seul nombre à la fois : il peut *conditionner* plusieurs valeurs dans un seul chiffré. Dans la version classique, on peut stocker environ $N/2$ nombres complexes dans un chiffré basé sur l'anneau R_q [?].

L'anneau des polynômes n'est pas vu seulement comme des coefficients, mais aussi comme des valeurs du polynôme en certaines racines de l'unité. On passe d'une représentation *par coefficients* à une représentation *par valeurs*.

3.2 Une écriture simplifiée de l'encodage

Définition 3.1 (Encodage dans CKKS). *CKKS encode un vecteur $\mathbf{z} = (z_1, \dots, z_{n/2}) \in \mathbb{C}^{n/2}$ en un polynôme $m(X) \in R$ tel que les évaluations de m aux racines primitives $2n$ -ièmes de l'unité approchent les composantes de $\mathbf{z}$.*

Si l'on note σ l'application de d'encodage qui transforme un vecteur de nombres complexes en un polynôme qui approxime les valeurs de $\mathbf{z}$ aux racines primitives $2n$ -ièmes de l'unité, alors l'encodage s'écrit schématiquement :

$$\text{Encodage}_\Delta(\mathbf{z}) = \lfloor \Delta \cdot \sigma(\mathbf{z}) \rfloor \in R,$$

où :

- $\mathbf{z}$ est un vecteur de nombres à chiffrer ;
- Δ est l'échelle ;
- $\lfloor \cdot \rfloor$ désigne l'arrondi coefficient par coefficient ;

— $R = \mathbb{Z}[X]/(X^N + 1)$ est la version entière de l'anneau.

Concrètement, σ est une interpolation polynomiale (*proche d'une transformée de fourier discrète*).

Le point important est le suivant : *on encode avec un arrondi, donc on accepte une petite erreur dès le départ*. CKKS est donc, dès l'encodage, un schéma approximatif.

4 Description détaillée de CKKS

4.1 Aperçu général

CKKS peut être vu comme un schéma à quatre grandes étapes :

1. génération des clés ;
2. encodage et chiffrement ;
3. calcul homomorphe ;
4. déchiffrement et décodage.

Algorithm 1 Version simplifiée de CKKS

- 1: **Entrée** : un vecteur de nombres $\mathbf{z}$, une échelle Δ
 - 2: Générer une paire de clés (pk, sk)
 - 3: Calculer $m \leftarrow \text{Encodage}_\Delta(\mathbf{z})$
 - 4: Chiffrer m pour obtenir $\mathbf{c}$ le message chiffré
 - 5: Effectuer des additions et multiplications homomorphes sur $\mathbf{c}$
 - 6: Si nécessaire, appliquer relinearisation, rescaling et bootstrapping
 - 7: Récupérer le résultat : $m^* \leftarrow \text{Dec}(\mathbf{c})$
 - 8: Retourner $\text{Decode}(m^*/\Delta)$
-

4.2 Génération des clés

On travaille dans un anneau $R_q = \mathbb{Z}_q[X]/(X^N + 1)$.

1. On choisit une clé secrète s “petite”, c'est-à-dire un polynôme de faible norme, souvent à coefficients dans $\{-1, 0, 1\}$.
2. On choisit un polynôme aléatoire a dans R_q .
3. On choisit un petit bruit e .
4. On définit la clé publique par

$$pk = (a, b), \quad b = -as + e \pmod{q}.$$

L'idée est que la paire (a, b) cache s à cause du bruit e . La sécurité repose sur la difficulté du problème RLWE (*Ring Learning With Errors*).

Remarque 4.1. *Les coefficients des polynômes "petits" ($e, s, \dots$) suivent une loi normale.*

4.3 Encodage et chiffrement

Soit un message réel ou complexe $\mathbf{z}$. On commence par l'encoder :

$$m = \text{Encodage}_\Delta(\mathbf{z}).$$

Le résultat m est un polynôme à coefficients entiers, mais il représente approximativement les valeurs de $\mathbf{z}$.

Pour chiffrer m :

1. on choisit un polynôme aléatoire u très petit ;
2. on choisit deux bruits petits e_1 et e_2 ;
3. on calcule

$$c_0 = bu + e_1 + m \pmod{q}, \quad c_1 = au + e_2 \pmod{q}.$$

Le chiffré est alors le couple

$$\mathbf{c} = (c_0, c_1) \in R_q^2.$$

4.4 Déchiffrement

En remplaçant b par $-as + e$, on obtient :

$$c_0 + c_1s = (bu + e_1 + m) + (au + e_2)s = eu - asu + e_1 + m + asu + e_2s = m + (eu + e_1 + e_2s).$$

Le terme entre parenthèses est petit. Donc, après déchiffrement, on retrouve m à une petite erreur près.

Il suffit ensuite de décoder ce polynôme pour retrouver un vecteur de nombres approchés.

4.5 Addition homomorphe

Si

$$\mathbf{c} = (c_0, c_1) \quad \text{et} \quad \mathbf{d} = (d_0, d_1)$$

sont deux chiffrés, alors leur somme est définie coefficient par coefficient :

$$\mathbf{c} + \mathbf{d} = (c_0 + d_0, c_1 + d_1).$$

En effet, lors du déchiffrement :

$$(c_0 + d_0) + (c_1 + d_1) \cdot s = (c_0 + c_1s) + (d_0 + d_1s) \approx m + m'$$

Un point important est que l'addition n'augmente *presque pas* le bruit du chiffrement. Elle effectue la somme des 2 bruits gardant l'erreur assez faible.

4.6 Multiplication homomorphe

La multiplication est plus délicate. On définit la multiplication de deux chiffrés de taille 2 (*multiplication de polynômes*) par :

$$(c_0, c_1) \cdot (d_0, d_1) = (c_0d_0, c_0d_1 + c_1d_0, c_1d_1) := (x_0, x_1, x_2)$$

Pour déchiffrer, on effectue :

$$x_0 + x_1s + x_2s^2 = c_0d_0 + (c_0d_1 + c_1d_0)s + c_1d_1s^2 = (c_0 + c_1s)(d_0 + d_1s) \approx m \cdot m'.$$

ce qui correspond bien au produit des messages. Mais le problème est que le degré du chiffré augmente (*ici de 2 à 3*).

Relinéarisation. Après une multiplication homomorphe, on obtient un chiffré de la forme (c_0, c_1, c_2) correspondant à :

$$m \approx c_0 + c_1 s + c_2 s^2$$

Le terme en s^2 empêche un déchiffrement direct classique.

Pour résoudre ce problème, on utilise une *clé de relinéarisation*, notée (rk_0, rk_1) , qui est un chiffrement de s^2 :

$$rk_0 + rk_1 s \approx s^2$$

On remplace alors :

$$c_2 s^2 \approx c_2 (rk_0 + rk_1 s)$$

Ainsi :

$$m \approx (c_0 + c_2 rk_0) + (c_1 + c_2 rk_1) s$$

On définit donc un nouveau chiffré :

$$(c'_0, c'_1) = (c_0 + c_2 rk_0, c_1 + c_2 rk_1)$$

Vérifiant :

$$c'_0 + c'_1 \cdot s \approx c_0 + c_1 s + c_2 s^2 \approx m$$

Ce chiffré est de nouveau linéaire en s , ce qui permet de continuer les calculs homomorphes.

Rescaling. Dans CKKS, chaque message m est encodé avec une échelle Δ pour convertir des nombres réels en entiers modulaires :

$$\tilde{m} = \lfloor m \cdot \Delta \rfloor$$

Après une multiplication homomorphe :

$$\tilde{m}_1 \cdot \tilde{m}_2 \approx m_1 m_2 \cdot \Delta^2$$

L'échelle a donc doublé, ce qui peut provoquer saturation et perte de précision.

Le *rescaling* consiste à diviser le chiffré par Δ pour revenir à une échelle contrôlable :

$$c_{\text{rescaled}} = \left\lfloor \frac{c}{\Delta} \right\rfloor \approx m_1 m_2 \cdot \Delta$$

Le rescaling permet de garder les valeurs numériques sous contrôle tout en conservant la précision relative.

5 Bootstrapping

5.1 Le problème du bruit

Chaque opération homomorphe augmente le bruit. À terme, si le bruit devient trop grand, le déchiffrement échoue : le terme utile est dépassé par les erreurs.

On ne peut donc effectuer qu'un nombre fini d'opérations avant de perdre la précision ou la validité du déchiffrement.

5.2 Principe du bootstrapping

Le *bootstrapping* est une méthode de “rafraîchissement” des chiffrés. L’idée, héritée de Gentry, est de faire calculer par le système homomorphe une version approximative de sa propre fonction de déchiffrement, afin de produire un nouveau chiffré contenant moins de bruit [?, ?].

Dans CKKS, cette étape est particulièrement subtile, car il faut :

- gérer le bruit déjà présent ;
- maintenir une bonne précision numérique ;
- réajuster l’échelle ;
- parfois utiliser des transformations de type FFT entre espace des coefficients et espace des slots.

Le bootstrapping permet donc d’enchaîner davantage d’opérations qu’un schéma purement limité, mais il reste coûteux.

6 Limites

6.1 Précision approximative

CKKS ne donne pas un résultat exact. À la place, il fournit une approximation contrôlée. Cela convient à beaucoup d’applications, mais pas à toutes. Un problème exact n’est pas le bon cadre pour CKKS.

6.2 Coût computationnel

Par rapport à un calcul non chiffré, CKKS est très coûteux :

- les polynômes sont de très grand degré ;
- les modules ont des tailles importantes ;
- les multiplications demandent des opérations de réécriture, de relinéarisation et de rescaling ;
- le bootstrapping est encore plus lourd.

Cependant, CKKS permet des calculs plus rapides que des méthodes de calculs homomorphes exactes (DGHV/TFHE) notamment grâce à ses calculs vectorisés.

6.3 Gestion du bruit

La gestion du bruit est le cœur pratique du schéma. Il faut choisir :

- la taille de l’échelle Δ ;
- le nombre de niveaux autorisés ;
- la taille du module q ;
- les paramètres de sécurité ;
- la stratégie de bootstrapping si elle est nécessaire.

Il existe donc un compromis permanent entre sécurité, précision et vitesse.

7 Conclusion

CKKS est aujourd’hui l’un des schémas homomorphes approximatifs les plus importants. Son originalité est d’accepter volontairement une petite erreur numérique afin de permettre des

calculs efficaces sur des nombres réels ou complexes. Cette idée est très pertinente en pratique, notamment pour les calculs statistiques et l'apprentissage automatique privé.

Du point de vue mathématique, CKKS repose sur l'arithmétique modulaire ainsi que les anneaux de polynômes. Du point de vue algorithmique, il introduit des opérations homomorphes bien structurées, mais qui nécessitent une gestion de l'échelle et du bruit.